

Concurrent Programming (CC3040)

University of Porto

Practical Assignment

Maximiliano Sá - up202305979 Pedro Machado - up202307196

May 2026

Contents

1	Exercise 1: Sending money using locks	3
1.1	Ex. 1.1: Two participants using naive locks	3
1.2	Ex. 1.2: Two participants with no deadlock	4
1.2.1	Proof of implementations 1.1 and 1.2 being not being weakly bisimilar	5
1.3	Ex. 1.3: Three participants using naive locks	6
1.4	Ex. 1.4: Generalisation of <i>Ex. 1.3</i> with value passing	6
2	Exercise 2: Simulate dependent processes in a server	8
2.1	Ex. 2.1: A thread-safe counter	8
2.2	Thread Pool Implementation	10
2.3	Ex. 2.2: Using lock-free programming to store the result of print instructions with timestamps	11
2.4	Ex. 2.3: Processing full instructions with the "after" clause	15
2.5	Ex. 2.4: Didactic functionality using <i>@volatile</i>	19
3	Exercise 3: Selling tickets with actors	22
3.1	Ex. 3.1: Implementation of the Ticket Office	22
3.2	Ex. 3.2: Ticket Office with a delegation functionality	22
3.2.1	Actor Hierarchy Diagram	24
3.2.2	Sequence Diagram	24

1 Exercise 1: Sending money using locks

1.1 Ex. 1.1: Two participants using naive locks

In this model, two participants try to send money repeatedly to each other. Each participant must acquire two locks (*lockA* and *lockB*) before being allowed to send money and must release them after sending the money.

This is an unsafe locking pattern that may lead to deadlocks. A problematic execution occurs when participant *A* acquires *lockA*, while participant *B* acquires *lockB*. At this point, both participants are stuck waiting for their other respective lock, held by the other participant, so neither can proceed, and they enter a deadlock state.

The model is composed of the following four parallel processes:

- *UserA*, which represents the first participant;
- *UserB*, which represents the second participant;
- *LockA*, which controls exclusive access to *lockA*;
- *LockB*, which controls exclusive access to *lockB*.

Implementation:

```
UserA := acquireLockA!.acquireLockB!.sendAToB!.releaseLockB!.releaseLockA!.UserA
UserB := acquireLockB!.acquireLockA!.sendBToA!.releaseLockA!.releaseLockB!.UserB

LockA := acquireLockA?.releaseLockA?.LockA
LockB := acquireLockB?.releaseLockB?.LockB

(UserA|UserB|LockA|LockB)\{acquireLockA,releaseLockA,acquireLockB,releaseLockB}
```

Figure 1: Implementation of exercise 1.1

The synchronization actions, *acquireLockA*, *releaseLockA*, *acquireLockB*, and *releaseLockB* are hidden by restriction, so they are only used internally to coordinate the users and the locks. The visible actions, *sendAToB* and *sendBToA*, in the model are the money-transfer actions, which occur only after both locks have been acquired. After a visible action has been executed, the locks held by the communicating user are freed for future use by either of the two participants.

LTS:

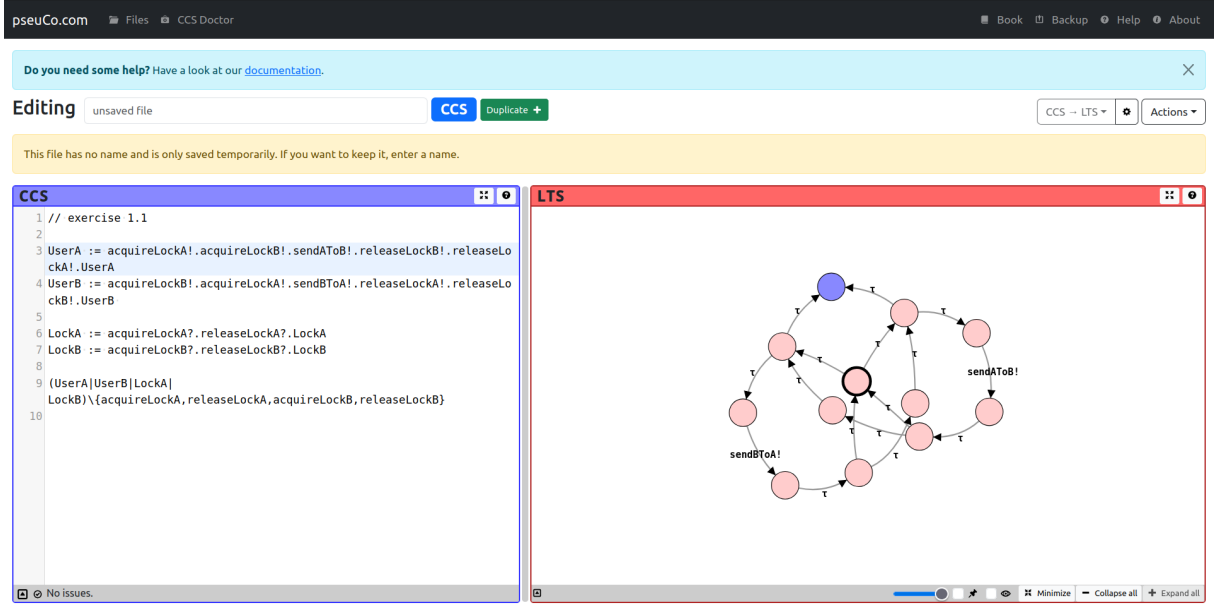


Figure 2: LTS exercise 1.1.

1.2 Ex. 1.2: Two participants with no deadlock

This model uses the same structure as *exercise 1.1*, but now both participants acquire the locks in the same order: *lockA* and then *lockB*.

With lock ordering, we ensure that, for example, when participant A waits to acquire lock *X* that is already acquired by participant *B*, participant *B* will never try to acquire any lock *Y* held by A such that $Y < X$ and will only try to acquire locks with ordering greater than *X*. As a result, the system no longer reaches a deadlock state.

Implementation:

```
UserA := acquireLockA!.acquireLockB!.sendAToB!.releaseLockB!.releaseLockA!.UserA
UserB := acquireLockA!.acquireLockB!.sendBToA!.releaseLockB!.releaseLockA!.UserB

LockA := acquireLockA?.releaseLockA?.LockA
LockB := acquireLockB?.releaseLockB?.LockB

(UserA|UserB|LockA|LockB)\{acquireLockA,releaseLockA,acquireLockB,releaseLockB}
```

Figure 3: Implementation of exercise 1.2

This implementation uses the same synchronization and visible actions as in *exercise 1.1*.

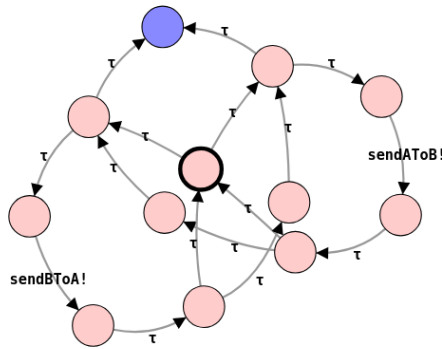
1.2.1 Proof of implementations 1.1 and 1.2 being not being weakly bisimilar

The systems implemented in exercises 1.1 and 1.2 are not weakly bisimilar. The LTS of the systems are found in Figure 4a for exercise 1.1 and in Figure 4b for exercise 1.2. Note that the blue state in Figure 4a corresponds to a deadlock state as no actions are allowed from it, and also the initial state in both LTS is identified with a black border.

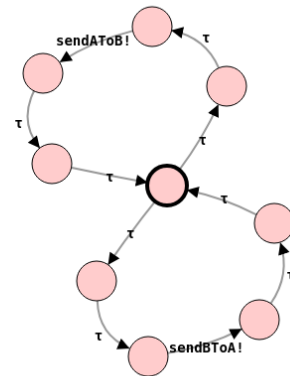
To prove that the two systems are not weakly bisimilar, we show that the attacker in the weak bisimulation game has a strategy that guaranties a win regardless of the moves done by the defender. The strategy is the following:

- **1.** The attacker starts from the initial state in the LTS of exercise 1.1, and at the first two plays, he will choose two τ actions that will lead him to the deadlock state (shown in blue in Figure 4a).
- **2.** After that, the attacker chooses to change positions with the defender. Now, the defender can only respond with weak τ transitions in the LTS of exercise 1.1. Therefore, the attacker, now in one of the states in 4b, will continue to choose τ transitions until it reaches a state that allows a *send* action.
- **3.** When the attacker arrives at one of the states that allows a *send* action, it will attack with a *send* and the defender will not be able to respond with a weak *send* action, as it is in the blue deadlock state in the LTS of exercise 1.1.

This winning strategy from the attacker works for any moves that the defender can do, and with that we can conclude that the two systems are not weakly bisimilar.



(a) LTS exercise 1.1



(b) LTS exercise 1.2

Figure 4

1.3 Ex. 1.3: Three participants using naive locks

This model was extended to three participants with different communication rules:

- Zenitsu (*UserZ*) can only send to Hinata (*UserH*).
- Hinata (*UserH*) can only send to Eren (*UserE*).
- Eren (*UserE*) can send to Zenitsu (*UserZ*) and Hinata (*UserH*).

The same locking pattern in *exercise 1.1* is used to control access to communication actions. Each user process is specialized according to whom they are allowed to communicate with.

Implementation:

```
UserZ := acquireLockZ!.acquireLockH!.sendZToH!.releaseLockH!.releaseLockZ!.UserZ
UserH := acquireLockH!.acquireLockE!.sendHToE!.releaseLockE!.releaseLockH!.UserH
UserE := acquireLockE!.(acquireLockH!.sendEToH!.releaseLockH!.releaseLockE!.UserE +
acquireLockZ!.sendEToZ!.releaseLockZ!.releaseLockE!.UserE)

LockZ := acquireLockZ?.releaseLockZ?.LockZ
LockH := acquireLockH?.releaseLockH?.LockH
LockE := acquireLockE?.releaseLockE?.LockE

(UserZ|UserH|UserE|LockZ|LockH|LockE)\{acquireLockZ,releaseLockZ,acquireLockH,releaseLockH,acquireLockE,releaseLockE}
```

Figure 5: Implementation of exercise 1.3

1.4 Ex. 1.4: Generalisation of *Ex. 1.3* with value passing

This model generalises the scenario of 3 participants sending money as described above in *exercise 1.3*, allowing each participant to send messages to any other participant. Instead of writing a separate process for each participant, a single parametrised process **Send**[*x*,*y*] is used to represent any participant sending a message from *x* (sender) to *y* (receiver).

The lock process is also parametrised, making the model compact and reusable. The structure of the lock model is still the same as in the previous exercises, but now the communication pattern is fully generalised.

The following CCS expression represents the global system configuration for three participants (0, 1, 2) who can send messages to any of the other participants:

```
1 (Send [0, 1] | Send [0, 2] | Send [1, 0] | Send [1, 2] | Send [2, 0] |
2 Send [2, 1] | Lock [0] | Lock [1] | Lock [2]) \{acquireLock, releaseLock}
```

Implementation with deadlocks:

```
Send[x,y] := acquireLock!x.acquireLock!y.send!x.receive!y.releaseLock!y.releaseLock!x.Send[x,y]
Lock[x] := acquireLock?x.releaseLock?x.Lock[x]

(Send[0,1]|Send[0,2]|Send[1,0]|Send[1,2]|Send[2,0]|Send[2,1]|Lock[0]|Lock[1]|Lock[2])\{acquireLock,releaseLock}
```

Figure 6: Implementation of exercise 1.4 with deadlocks

In the above implementation, each participant acquires locks according to the order defined by the sender and receiver identifiers. Since different processes may attempt to acquire the same pair of locks in different orders, each process will wait indefinitely for the second lock held by the other process, thus entering into a deadlock. The global system configuration is obtained through the parallel composition of all sender processes and all lock processes, while restricting the internal lock synchronisation actions.

Deadlock scenario: *user0* attempts to send a message to *user1* while *user1* attempts to send a message to *user0*. *user0* will acquire *lock0* and *user1* will acquire *lock1*. *user0* will now attempt to acquire *lock1*, while *user1* attempts to acquire *lock0*. The locks that both processes are trying to acquire are already held by each of the other respective processes, therefore the two users become stuck indefinitely, thus reaching a deadlock state.

Implementation without deadlocks:

```
Send[x,y] := when(x<y)acquireLock!x.acquireLock!y.send!x.receive!y.releaseLock!y.releaseLock!x.Send[x,y] + when(x>y)acquireLock!
y.acquireLock!x.send!x.receive!y.releaseLock!x.releaseLock!y.Send[x,y]
Lock[x] := acquireLock?x.releaseLock?x.Lock[x]

(Send[0,1]|Send[0,2]|Send[1,0]|Send[1,2]|Send[2,0]|Send[2,1]|Lock[0]|Lock[1]|Lock[2])\{acquireLock,releaseLock}
```

Figure 7: Implementation of exercise 1.4 without deadlocks

The above implementation solves the deadlock problem by enforcing a global ordering on the lock acquisition. Before acquiring locks, each process compares the participant's identifiers and always acquires the lock with the smaller identifier first.

When $x < y$, the process acquires *lockX* before *lockY*, otherwise, it acquires *lockY* before *lockX*. This strategy removes the possibility of circular waiting, since no two processes can hold locks in conflicting orders. Consequently, the system no longer reaches deadlock states while preserving the intended communication between participants.

Going back to the scenario from the first implementation of *exercise 1.4*: *user0* attempts to send a message to *user1* while, simultaneously, *user1* attempts to send a message to *user0*. Since $0 < 1$, both processes follow the same order of lock acquisition. Assuming

that *user0* acquires *lock0* first, *user1* will have to wait for *lock0* to be released. *user0* then successfully acquires *lock1*, sends a message to *user1*, and releases both locks in reverse order. *user1* can now acquire the freshly released *lock0* and *lock1*, send a message to *user0*, and then release both locks in reverse order.

Therefore, both users communicate with each other without ever entering a deadlock.

2 Exercise 2: Simulate dependent processes in a server

2.1 Ex. 2.1: A thread-safe counter

The original counter uses a shared mutable integer:

```
1 var counter = 0;
```

This counter, initially, is not thread-safe because its increment when a new process arrives is not done atomically. As the server processes requests concurrently, multiple threads may read the same counter value simultaneously before any of them writes the updated value. Therefore, some increments are lost. The following input example shows this loss of increments:

```
1 server: localhost:8080
2 print "hello"
3 server: localhost:8080
4 print "bye"
5 server: localhost:8080
6 print "hello"
7 server: localhost:8080
8 print "bye"
9 server: localhost:8080
10 print "hello"
```

Note that the above input has been shortened for the purposes of the report. The original input file contains 200 requests from different client connections. The intended counter output is 200.

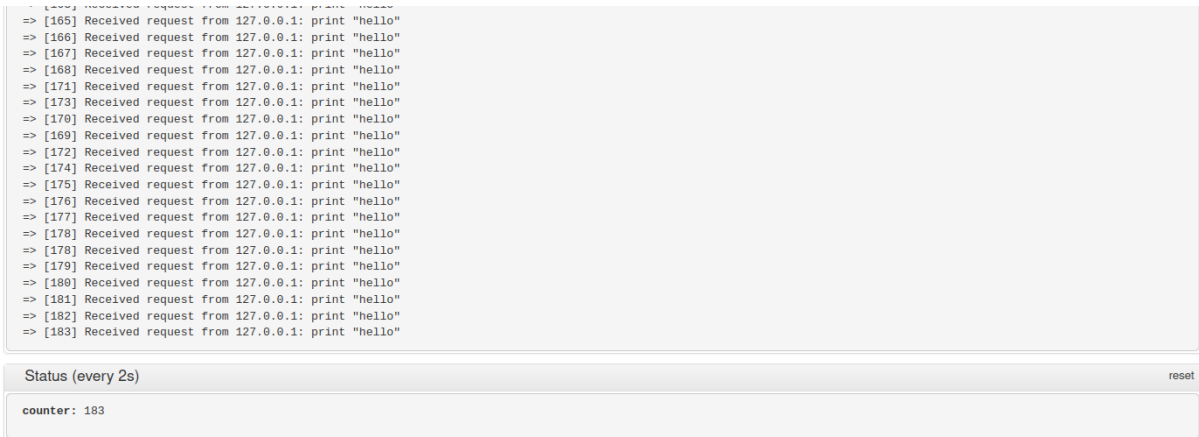


Figure 8: Output without a thread-safe counter

As seen in the figure above, the non-thread-safe counter missed 17 increments. The solution is to use *AtomicInteger* from the *java.util.concurrent.atomic* library.

```

1 import java.util.concurrent.atomic._
2 var counter = new AtomicInteger();

1 private def runProcess(cmd: String, userIp: String): String = {
2   val cnt = state.counter.incrementAndGet()
3   ...

```

The *AtomicInteger* class provides atomic operations such as *incrementAndGet()*, which was used in our *runProcess()* function, which guaranties that concurrent increments are performed safely without lost updates. The situation in which two different threads read the same counter value and then increment it, will not happen under an Atomic Variable because reads and increments to it are done atomically. This implementation is lock-free and efficient.

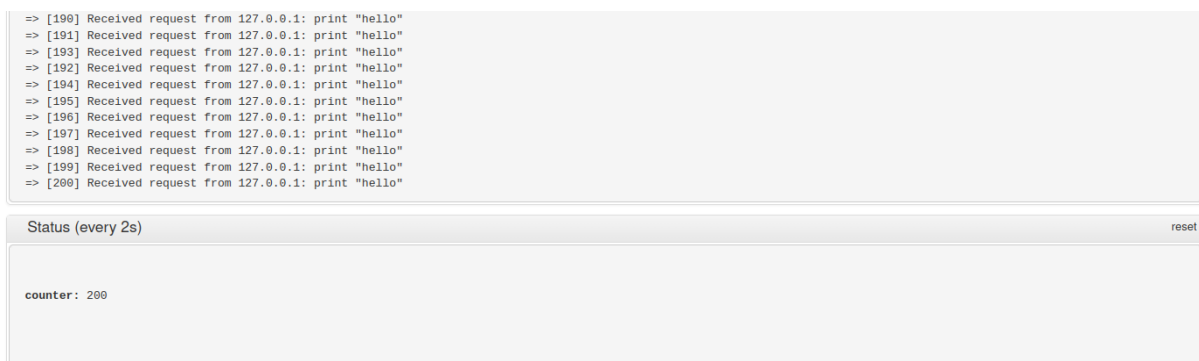


Figure 9: Output with a thread-safe counter

The above output of 200 shows that no increments are lost when using *AtomicInteger*.

2.2 Thread Pool Implementation

A Thread Pool (*ThreadPool.scala*) was implemented to manage the execution of the instructions.

```
1      class ThreadPool(val num_threads: Int) {  
2          private val tasks = mutable.Queue[() => Unit]()
```

The shared queue stores tasks waiting to be executed.

The implementation creates a fixed number of worker threads that continuously fetch and execute tasks from the shared queue.

```
1      for (_ <- 0 until num_threads) {  
2          val w = new Worker  
3          w.start()  
4      }
```

For the purpose of this project, 4 threads were used.

```
1      val thread_pool: ThreadPool = new ThreadPool(4)
```

Whenever a new instruction is received by the server, it is inserted into the queue using the *execute()* method.

```
1      def execute(task: => Unit): Unit = {  
2          tasks.synchronized {  
3              tasks.enqueue(() => task)  
4              tasks.notify()  
5          }  
6      }
```

The queue is protected using *synchronized* in order to enforce mutual exclusion when adding or removing a task. After inserting a task, *notify()* is called to wake up a worker thread waiting for work.

Each worker thread executes in the following steps:

1. wait until a task is available
2. remove task from the queue
3. execute the task

This execution logic is seen below:

```

1  def poll() = {
2      tasks.synchronized {
3          while (tasks.isEmpty) {
4              tasks.wait()
5          }
6          tasks.dequeue()
7      }
8  }

```

wait() allows worker threads to sleep while the queue is empty to avoid busy waiting and unnecessary CPU usage.

Worker threads continuously execute tasks using an infinite while loop in *run()*.

```

1  override def run() = {
2      while (true) {
3          val task = poll()
4          task()
5      }
6  }

```

2.3 Ex. 2.2: Using lock-free programming to store the result of print instructions with timestamps

In the *ServerState()* class, the thread-safe structure using lock-free programming that is used to store the result and timestamp of completed instructions was implemented with an *AtomicReference* to a *List[String]*. The functions to add a new output and reset its state are *addResult* and *clearResults*, respectively.

Each element of the list stores the output of a completed instruction and the time stamp of when it finished executing.

```

1  var results = new AtomicReference[List[String]](Nil);

```

To safely update the shared structure concurrently, the implementation uses the lock-free operation *compareAndSet*, in the *addResultMethod* function which repeatedly attempts to update the shared list until the operation succeeds.

```

1  def addResult(msg: String): Unit = {
2      var update = false;
3
4      while (!update){

```

```

5         val cur = results.get()
6
7         val newList = msg :: cur
8
9         update = results.compareAndSet(cur, newList)
10    }
11}

```

The current list is retrieved from the atomic reference and a new list containing the freshly executed instruction is created. *compareAndSet* check if the atomic reference has changed before writing to it. If the atomic reference has not changed since being fetched *compareAndSet* writes the new list to the atomic reference, otherwise, it starts the fetch-create-write process again.

The most recently executed instruction is placed at the head of the list, therefore when outputting the instructions in the *toHtml* function the list is reversed.

```

1    val out = results.get().reverse.mkString("<br>")

```

A function *clearResults()* was used to clear the list when the user requests to clear the output.

```

1    def clearResults(): Unit = {
2        results.set(Nil)
3    }

```

When the server receives a request, the instructions in the request will be executed concurrently by the thread pool. Before fetching the time stamp and storing the result of some instruction, the thread that is executing, that same instruction, will stop its execution for *delay* second, in order to simulate this instruction.

```

1    for(cmd <- cmds){
2        val (instruction, delay) = parseIgnoreAfter(cmd)
3
4        thread_pool.execute({
5            if (delay > 0){
6                Thread.sleep(delay)
7            }
8
9            val timestamp = LocalDateTime.now()
10           state.addResult(s"[$timestamp] : ${instruction}")
11       })

```

A custom parser function, *parseIgnoreAfter*, that ignores the after clause, was created specifically for the implementation of this exercise. First, comments are removed from the received instruction (*rawCommand*) instruction. Then the instruction is split at the word "after" and the prefix is used. The prefix is then split at the '@'. If there is a number after the split then it is trimmed and converted to an integer using *toInt* and stored in *delay*. The left side of the split is the instruction which is then split at the first quotation mark to remove *print* and split again to remove the last quotation mark. The output and delay are then returned by the function. Note that the delay is multiplied by 1000 to convert it to milliseconds for *Thread.sleep*.

```

1      private def parseIgnoreAfter(rawCommand: String): (String,
2          Int) = {
3          val noComment = rawCommand.split("#", 2).head.trim
4
5          val prefix = noComment.split("\\bafter\\b", 2).head.trim
6
7          val parts = prefix.split("@", 2)
8          val instruction = parts(0).trim
9          val delay = if (parts.length == 2 && parts(1).nonEmpty)
10             parts(1).trim.toInt else 0
11
12         val output = instruction.split("\"", 2)(1).split("\"", 2)
13             (0)
14
15         (output, delay * 1000) // converting seconds to
16             milliseconds for thread.sleep
17     }

```

Input program that would likely fail with a non-thread safe implementation:

The following input would likely fail without a thread-safe implementation of the shared structure since each instruction would execute simultaneously on different worker threads and try to write to the *results* list stored in the server state. Without *AtomicReference* and, compare and set, it is highly likely that some writes to the shared structure would be lost.

```

1      server: localhost:8080
2      print "A" @ 1
3      print "B" @ 1
4      print "C" @ 1

```

```

5      print "D" @ 1
6      print "E" @ 1
7      print "F" @ 1
8      print "G" @ 1
9      print "H" @ 1
10     print "I" @ 1
11     print "J" @ 1
12     print "K" @ 1
13     print "L" @ 1
14     print "M" @ 1
15     print "N" @ 1
16     print "O" @ 1
17     print "P" @ 1

```

Using the above lock-free thread-safe implementation of the shared structure the following result would be obtained:

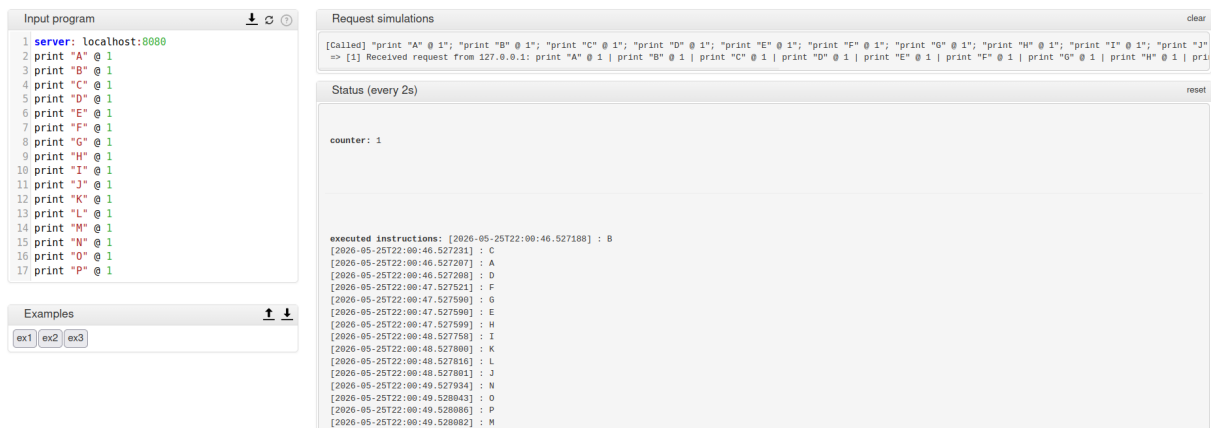


Figure 10: Output using a thread-safe structure using lock-free programming

Clear server state:

The following snippet displays the sever call to reset the counter and the output results list.

```

1      case GET -> Root / "reset" =>
2          state.counter.getAndSet(0)
3          state.clearResults()
4          Ok("State reset!")
5          .map(addCORSHeaders)

```

The variables *counter* and *results* are atomic, but changes in both variables are not an atomic operation. For that reason, the following situation can occur:

- The user starts by sending a reset message. However, between the instruction that resets the *counter* to 0 and the instruction that resets the *results* list, the server sends its state back to the user. The problem is that the user would receive the counter with value 0 and the *results* list would be the one before the reset. This situation is not critical because the server would eventually reset the list. One solution to this problem would be to create an object that stores both the *counter* and *results* lists, and a change to this object would be performed atomically.

2.4 Ex. 2.3: Processing full instructions with the "after" clause

In this exercise, support for the *after* clause was included. Each instruction may specify one or more instructions that must be completed before it is allowed to be executed. The execution uses the same execution logic as *exercise 2.2* along with the following changes:

An instruction class, *Instr*, was created.

```
1 private case class Instr(output: String, delay: Int,  
    afterDependencies: List[Int])
```

The list of commands, *cmds*, is stored in an array, *parsedInstr*, after mapping the parser function to each one of them.

```
1 val parsedInstr = cmds.map(parseAll)
```

The parser function, *parseAll()* used in this exercise uses the same logic as the parser function in the previous exercise except that we do not discard the after part of the instruction. The after dependencies are stored in a list and *parseAll()* returns a *Instr* object. Note: instruction numbers were converted to base-0 by subtracting one for simpler array indexing.

```
1 private def parseAll(rawCommand: String): Instr = {  
2     val noComment = rawCommand.split("#", 2).head.trim  
3  
4     val afterSplit = noComment.split("\\bafter\\b", 2).map(_.trim  
5         )  
6     val leftSide = afterSplit(0)  
7     val afterPart = if (afterSplit.length == 2) Some(afterSplit  
        (1)) else None
```

```

8     val afterDependencies: List[Int] = afterPart match {
9         case None => Nil
10        case Some(s) => if (s.isEmpty) Nil else s.split(",").toList
11            .map(_ .trim).map(_ .toInt).map(_ - 1)
12    }
13
14    val atSplit = leftSide.split("@", 2)
15    val instruction = atSplit(0).trim
16    val delay = if (atSplit.length == 2 && atSplit(1).nonEmpty)
17        atSplit(1).trim.toInt else 0
18
19    val output = instruction.split("\\"", 2)(1).split("\\"", 2)(0)
20
21    Instr(output = output, delay = delay * 1000,
22        afterDependencies = afterDependencies)
23 }

```

The dependency mechanism is based on:

1. *dependencies*: a dependency graph storing which instructions depend on each instruction.
2. *remaining*: storing how many dependencies are still unfinished for each instruction. The values of each element in this array are decremented by multiple threads concurrently, therefore, to avoid inconsistent values, it is an array of Atomic Integers.
3. *no_deps_instr*: For each instruction, tells whether it has any initial dependencies.

```

1     val dependencies: Array[List[Int]] = Array.fill(parsedInstr.
2         size)(Nil)
3     val remaining: Array[AtomicInteger] = Array.ofDim[
4         AtomicInteger](parsedInstr.size)
5     val no_deps_instr: Array[Boolean] = new Array[Boolean](
6         parsedInstr.size)

```

The following function, *executeInstruction*, was used to execute an individual instruction:

```

1     def executeInstruction(i: Int): Unit = {
2         val instruct = parsedInstr(i)
3
4         thread_pool.execute {

```

```

5      if (instruct.delay > 0){
6          Thread.sleep(instruct.delay)
7      }
8
9      val timestamp = LocalDateTime.now()
10     state.addResult(s"[${timestamp}] : ${instruct.output}")
11
12     dependencies(i).foreach{j =>
13         val left = remaining(j).decrementAndGet()
14         if (left == 0){
15             executeInstruction(j)
16         }
17     }
18 }
19 }

```

The index of the instruction is received and the corresponding instruction is fetched from *parsedInstr*. The thread executing the instruction then sleeps for the delay of the instruction, if one exists, and then generates the time stamp of execution. The *addResult* function described in the previous exercise is used to store the output with its respective time stamp in the server state. The thread then atomically decrements the number of remaining unresolved dependencies for each of the instructions that depend on the one that just executed. If the dependency counter of any dependant instruction reaches zero, it becomes ready for execution and is immediately scheduled through the *executeInstruction* function.

Initially, instructions with no dependencies are executed first as seen in the following snippet:

```

1  parsedInstr.indices.foreach{ i =>
2      if (no_deps_instr(i)) {
3          executeInstruction(i)
4      }
5  }

```

This approach is more CPU efficient in comparison to the naive approach in which each instruction has to busy-wait until all its dependencies have finished.

3 Inputs illustrating that dependency mechanism is working:

The following three inputs demonstrate the functionality of the dependency mechanism described above.

Input 1:

```
1 server: localhost:8080
2 print "hello, i am pedro" @ 3 after 3
3 print "sup, i am bob" after 1,3
4 print "hi, i am max" @ 1
```

Output of input 1:

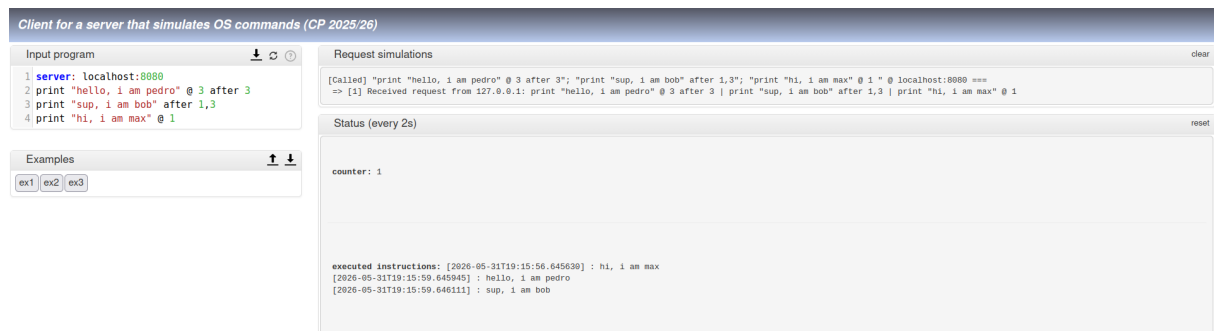


Figure 11: Output of input 1 for exercise 2.3

Input 2:

```
1 server: localhost:8080
2 print "hello, i am pedro" @ 3 after 3
3 print "sup, i am bob" after 1,3,4,5
4 print "hi, i am max" @ 1
5 print "how are you guys?" @ 2 after 1
6 print "max is good" @ 1 after 4
```

Output of input 2:

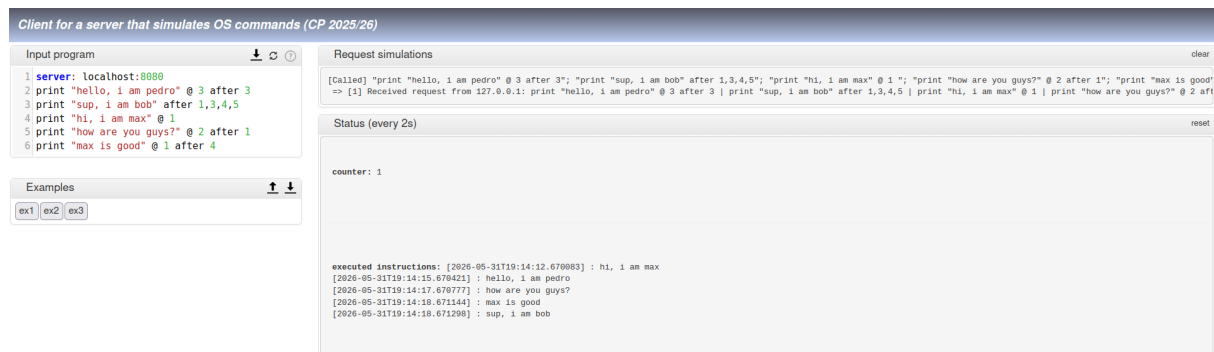


Figure 12: Output of input 2 for exercise 2.3

Input 3:

```
1 server: localhost:8080
2 print "1 slow (4s)" @ 4
3 print "2 fast (1s)" @ 1
4 print "3 from fast (1s)" @ 0 after 2
5 print "4 from slow (4s)" @ 0 after 1
6 print "5 final (from 3 and 4)" @ 0 after 3,4
```

Output of input 3:

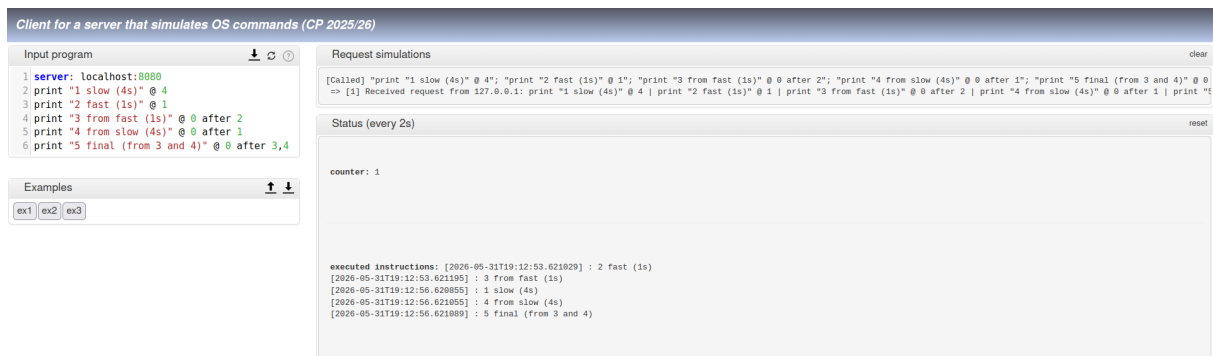


Figure 13: Output of input 3 for exercise 2.3

In a real world server, it would be necessary to add some mechanism that checks the existence of a dependency cycle formed by the instructions in a request. If a dependency cycle between instructions is not detected, the server will not be able to complete tasks. In our implementation, we assume that in every request sent by the user, its instructions dependencies form a Direct Acyclic Graph (DAG).

2.5 Ex. 2.4: Didactic functionality using *@volatile*

For this exercise, the server was extended with a pause and resume mechanism implemented by using a shared *@volatile* boolean variable.

```
1 @volatile private var paused: Boolean = false
```

Two additional HTTP routes were implemented to support this functionality.

```
1 case GET -> Root / "pause" =>
2     paused = true
3     Ok("SERVER PAUSED").map(addCORSHeaders)
4
```

```

5     case GET -> Root / "resume" =>
6         paused = false
7         Ok("SERVER RESUMED").map(addCORSHaders)

```

Whenever a thread begins executing an instruction, it first checks whether the server is currently paused.

```

1     while (paused){
2         Thread.sleep(1)
3     }

```

If the server is paused, the thread waits until the value of *paused* becomes *false*. Once the server is resumed, execution of the instructions continues normally.

The annotation `@volatile` is required to guaranty visibility across threads and to ensure that worker threads always retrieve the most recent value.

Suppose that the following sequence of events occurs:

1. a thread executing an instruction reads *paused == true*
2. Meanwhile, the server executes the *resume* route and changes the value of *paused* to *false*
3. the thread continues using its cached value and never observes the change in *paused*

In this situation, the thread may remain indefinitely inside the loop even though the server has already resumed.

`@volatile` prevents this issue by ensuring that every read of the variable retrieves the most recent value, and, consequently, all threads executing concurrently observe pause and resume operations.

Usage example of the pause/resume functionality

Suppose the following input:

```

1     server: localhost:8080
2     print "A" @ 30
3     print "B" @ 5 after 1
4     print "C" @ 5
5     print "D" @ 5

```

In order to test this functionality, during the first 30 seconds after sending the request above, via a web browser, insert the following URL:

1

```
http://localhost:8080/pause
```

counter: 1

executed instructions: [2026-05-31T18:40:31.289300] : C
[2026-05-31T18:40:31.289300] : D
[2026-05-31T18:40:56.289281] : A

Figure 14: Output after pausing

and after some time resume with:

1

```
http://localhost:8080/resume
```

counter: 1

executed instructions: [2026-05-31T18:40:31.289300] : C
[2026-05-31T18:40:31.289300] : D
[2026-05-31T18:40:56.289281] : A
[2026-05-31T18:48:08.479385] : B

Figure 15: Output after resuming

This will guaranty that instruction number 2 will not be able to execute immediately after instruction 1, because, if you successfully paused the server during the execution of instruction 1 and not resumed the server until the end of 1, instruction 2 will have to wait until the server is resumed again.

In Figure 15, it can be seen that the server was paused during the execution of instruction 1 and only resumed after 8 minutes, which was when instruction instruction 2 was finally able to continue its execution.

Without `@volatile`, some threads may not observe the resume operation and remain paused indefinitely, due to caching the `paused=true`. With `@volatile`, all threads correctly observe updates to the shared variable, `paused`, and continue execution normally after the server resumes.

3 Exercise 3: Selling tickets with actors

3.1 Ex. 3.1: Implementation of the Ticket Office

Implementation of an actor, supposed to simulate a Ticket Office, that starts with 0 *tickets* and reacts to the following messages:

- *ToSell*(*n*): the actor receives *n* tickets, adding *n* to the total number of *tickets*.
- *Buy*(*n*): the actor receives a message asking to buy *n tickets*. If the actor has enough *tickets* in stock to cover the buy request, it subtracts *n* from the total number of *tickets*; otherwise, it logs a message saying it could not provide *n* tickets.
- "Bye": stops the actor.

3.2 Ex. 3.2: Ticket Office with a delegation functionality

In this exercise, we created two actor classes: *SellerActor* and *ChildSellerActor*. A *SellerActor* is intended to serve as the main ticket office and should be capable of creating *ChildSellerActor* instances, assigning each one an initial fixed number of tickets whenever its own ticket stock exceeds a specified threshold.

A *SellerActor* starts with 0 *tickets* and is defined with a threshold value and delegation value, and reacts to the following messages:

- *ToSell*(*n*): It adds *n* to the number of *tickets* it has. After this addition, it checks if the number of available tickets is higher than the threshold, if this condition is verified, it creates and delegates a fixed amount of tickets to a new *ChildSellerActor*.

- *Buy(n)*: If it has more tickets than n , the purchase was successful and n is subtracted from *tickets*; otherwise a message is logged saying it could not provide that number of tickets. Additionally, in order to test its children with purchases, it sends a *Buy(n)* message to all its children.
- *Report(n)*: It receives this message when one of its children is unable to satisfy a buy request, sending all of its remaining tickets to be absorbed by its parent. As the number of tickets increases by n , it is necessary to perform the same procedure that is done after a *ToSell(n)* message, when it is checked the tickets threshold and the possibility of creating a new child ticket office.
- "Bye": The Main Office logs a message with the final amount of *tickets* it had in stock and sends a *Balance* message to all its children, for them to print their final amount of tickets. Finally, it stops its execution.

A *ChildSellerActor* can be in two states: alive and dead. When the actor is created by its parent, it is in state alive, only changing to the dead state after not being able to cover a buy request. When the actor is in state alive, after being created and starting with the number of *tickets* delegated by its parent, it is able to react to the following messages:

- *Buy(n)*: If it has more *tickets* in stock than n , the purchase was successful and the number of *tickets* available is decremented by n ; otherwise, the actor logs a message saying it could not cover the purchase, reporting to his parent with the aim of returning all its tickets in stock, and finally changes to dead state where it will simply ignore all the messages that receives.
- *Balance*: It logs the number of tickets currently in stock.

To test this implementation, the following code was used:

```

1 val sys = akka.actor.ActorSystem("TicketSys")
2
3 val ticketOffice = sys.actorOf(Props(new SellerActor(100,50)), "
   MainOffice")
4
5 ticketOffice ! ToSell(75)
6 ticketOffice ! Buy(100)
7 ticketOffice ! ToSell(75)
8 ticketOffice ! Buy(40)
9 ticketOffice ! Buy(70)
10
11 Thread.sleep(500)
12

```

```

13  ticketOffice ! ToSell(70)
14  ticketOffice ! Buy(20)
15  ticketOffice ! ToSell(50)
16  ticketOffice ! Buy(20)
17  ticketOffice ! Buy(15)
18
19  Thread.sleep(500)
20
21  ticketOffice ! "Bye"
22
23  Thread.sleep(10000)
24  sys.terminate()

```

3.2.1 Actor Hierarchy Diagram

In the code above, the actor system is named "TicketSys", and a *SellerActor* named "MainOffice" is created. "MainOffice", during the execution of the code, will create three children of *ChildSellerActor* class.

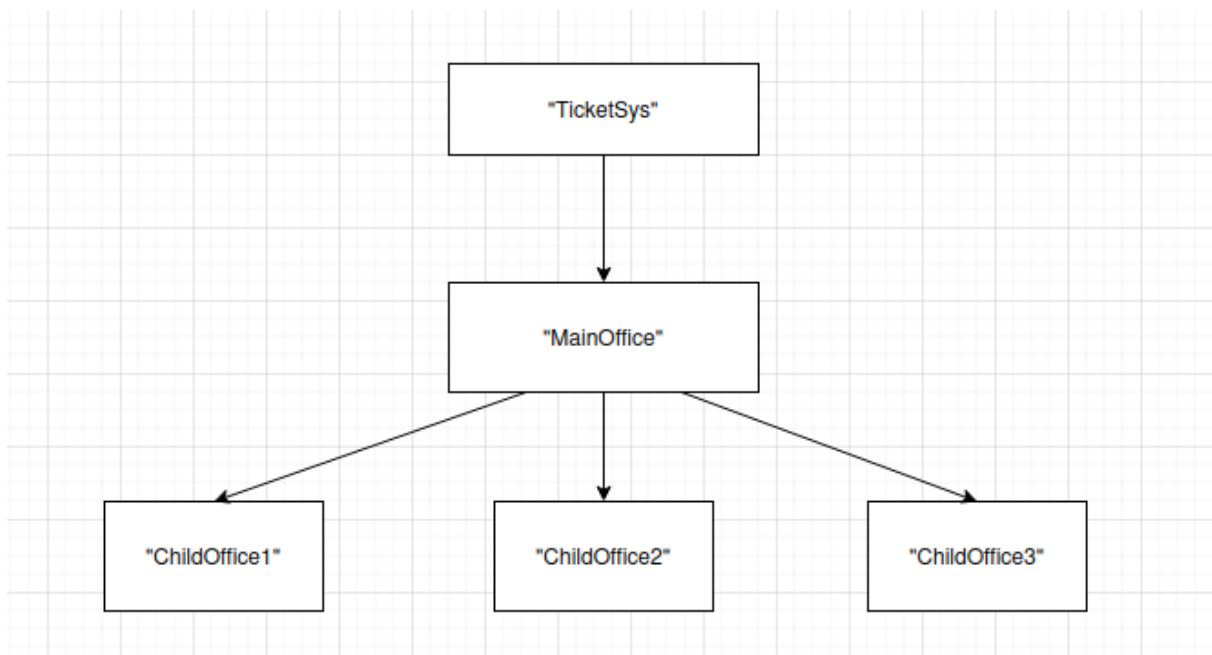


Figure 16: Actor Hierarchy Diagram

3.2.2 Sequence Diagram

This is the sequence message that occurs during the execution of the test code. To avoid another possible sequence of messages, two `Thread.sleep(500)` instructions were inserted to guaranty that the report message sent by one child, after not having enough

tickets for a purchase, is processed by its parent before the system sends others ToSell and Buy messages. A Thread.sleep(10000) instruction was inserted before the sys.terminate() instruction to guaranty that all the messages sent were processed.

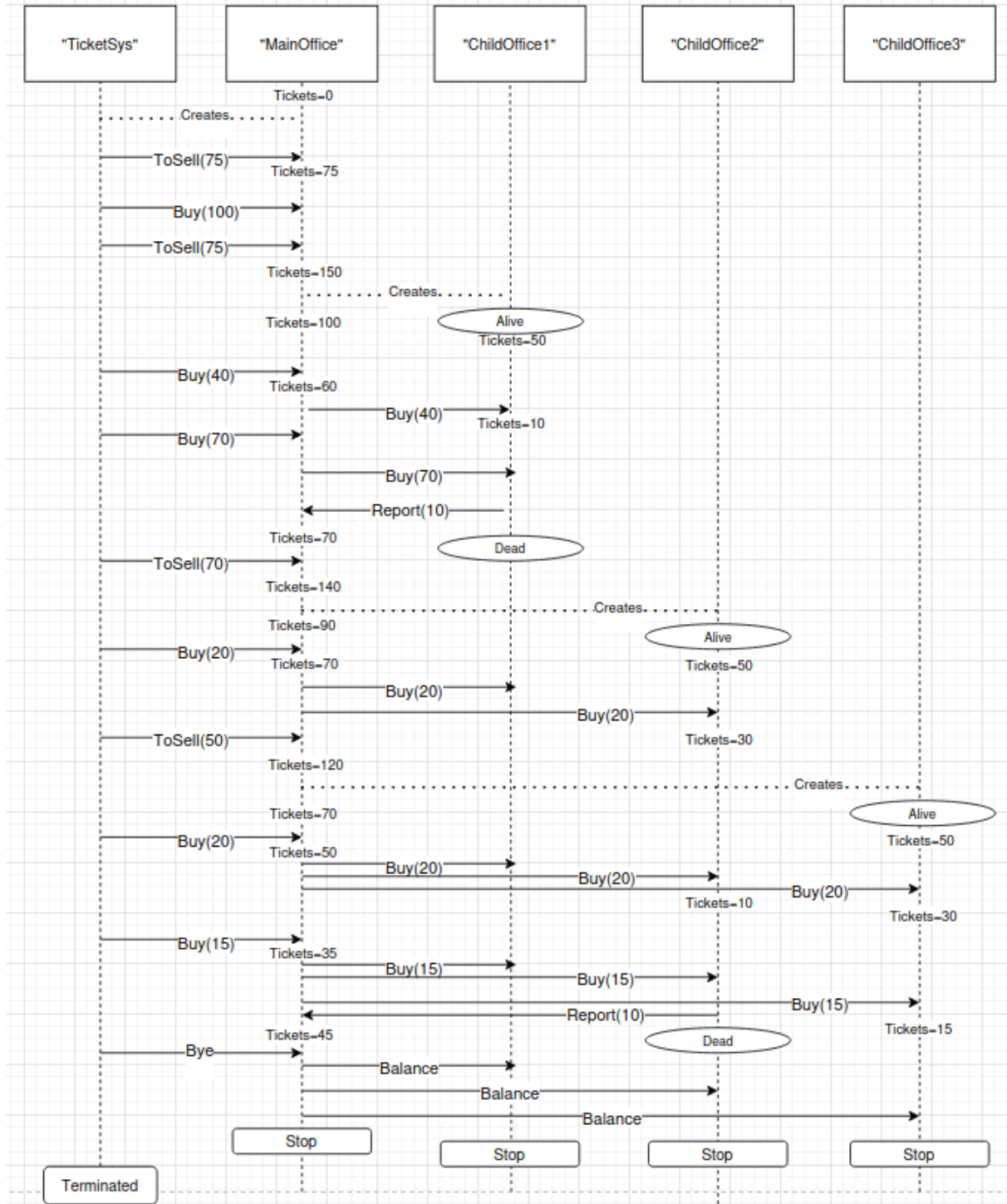


Figure 17: Sequence Diagram - Testing Code